

Multivariate rational function interpolation

by

Archit Srivastava

M.S., University of Nebraska, Lincoln 2022

B.Tech., Dr. Abul Kalam Azad Technical University, 2017

Thesis Submitted in Partial Fulfillment of the
Requirements for the Degree of
Master of Science

in the
Department of Mathematics
Faculty of Example Names

© **Archit Srivastava 2025**
SIMON FRASER UNIVERSITY
Fall 2025

Copyright in this work is held by the author. Please ensure that any reproduction
or re-use is done in accordance with the relevant national copyright legislation.

Declaration of Committee

Name: Archit Srivastava
Degree: Master of Science
Thesis title: Multivariate rational function interpolation
Committee: **Chair:** Pamela Isely
Assistant Professor, Computing Science

Emmett Brown
Supervisor
Professor, Computing Science

Bonnibel Bubblegum
Committee Member
Associate Professor, Computing Science

James Moriarty
Committee Member
Adjunct Professor, Computing Science

Kaylee Frye
Examiner
Assistant Professor, Engineering Science

Hubert J. Farnsworth
External Examiner
Professor
Department of Quantum Fields
Mars University

Abstract

Abstract paragraphs should be unindented. Master's abstracts are limited to 150 words; the limit is 350 words for doctoral abstracts. Abstract text must fit on a single page.

Keywords: thesis template; Simon Fraser University; L^AT_EX time travel paradoxes

Dedication

This is an optional page. Use your choice of paragraph style for text on this page.

Acknowledgements

This is an optional page. Use your choice of paragraph style for text on this page.

Table of Contents

Declaration of Committee	ii
Abstract	iii
Dedication	iv
Acknowledgements	v
Table of Contents	vi
List of Tables	viii
List of Figures	ix
1 Introduction	1
1.1 Blackboxes and Evaluation homomorphism	1
1.1.1 Blackboxes	1
1.1.2 Evaluation homomorphism	1
1.2 Randomized algorithm, Early termination	1
2 Maximal Quotient Rational Function Reconstruction	2
2.1 Univariate Rational function reconstruction	2
2.1.1 Interpolation	2
2.1.2 Extended Euclidean Algorithm	2
2.1.3 Chinese Remainder Theorem	3
2.1.4 Chinese remainder theorem and Interpolation	3
2.1.5 Main idea	4
3 Ben-Or Tiwari Interpolation	6
3.1 Introduction	6
3.2 Linear Recursive Sequences	6
3.2.1 Linear recurrence as Linear System	6
3.2.2 Characteristic Polynomial of the Companion Matrix	8

3.2.3	Eigen values of the Companion matrix are the general solution of the linear recurrence	10
3.2.4	Hankel Matrix	12
3.2.5	Polynomials as linear recurrences.	13
3.2.6	Hankel matrix with Gaussian elimination	15
3.2.7	Berlekamp Massey Algorithm	15
3.2.8	Equivalence of Berlekamp Massey Algorithm and Extended Euclidean Algorithm	15
3.3	Polynomials as vectors	15
3.4	Interpolation as a linear map	16
3.4.1	Zipple Transposed Vandermonde Solver	16
3.4.2	Fast Transposed Vandermonde Solver	16
3.5	Ben-Or and Tiwari's Multivariate Polynomial Interpolation	16
4	Multivariate Rational function interpolation	18
4.1	Kaltofen Yang algorithm	18
5	Applications	22
	Bibliography	23
	Appendix A Code	24

List of Tables

List of Figures

Chapter 1

Introduction

Univariate interpolation is an old problem with the first systematic occurrence

The methods for finding univariate polynomials fitting a set of points have been around since 16th Century with Newton's method for divided differences and

Given $d + 1$ points in an n dimensional space $\mathbb{Z}_p^n$, the objective is to find a multivariate rational function in $\mathbb{Z}_p[x_1, \dots, x_n]$

1.1 Blackboxes and Evaluation homomorphism

1.1.1 Blackboxes

Definition 1.1.1. *A blackbox is a function that takes an input and produces an output. The internal workings of the function are not known to the user. The user can only interact with the blackbox by providing inputs and observing the outputs.*

1.1.2 Evaluation homomorphism

Definition 1.1.2. *Let K be a field and $K[x] \in K(x)$ be the ring of polynomials and field of rational functions respectively. Let $\alpha \in K$ be a point in the field. The evaluation homomorphism is a map*

$$\begin{aligned}\phi: K[x] &\longrightarrow K[x]/(x - \alpha) \cong K \\ f(x) &\longmapsto f(\alpha)\end{aligned}$$

1.2 Randomized algorithm, Early termination

Chapter 2

Maximal Quotient Rational Function Reconstruction

2.1 Univariate Rational function reconstruction

We are given a black box for a rational function $F(x) = \frac{f(x)}{g(x)} \in K(x)$ where $f(x), g(x) \in K[x]$ and $g(x) \neq 0$. We need to recover the polynomials $f(x), g(x)$ upto a constant factor. We can evaluate the black box at n distinct points $\alpha_1, \dots, \alpha_n$ to get the values $y_1, \dots, y_n$.

2.1.1 Interpolation

Given a set of $k + 1$ distinct points we can find an interpolating polynomial of degree k using different polynomial interpolation methods, the most commonly used being Newton's method or Lagrange's method. The choice of method determines the basis of the polynomial.

2.1.2 Extended Euclidean Algorithm

Algorithm 1 Extended Euclidean Algorithm

```
1:  $r_0 \leftarrow f, \quad s_0 \leftarrow 1, \quad t_0 \leftarrow 0$ 
2:  $r_1 \leftarrow g, \quad s_1 \leftarrow 0, \quad t_1 \leftarrow 1$ 
3:  $i \leftarrow 1$ 
4: while  $r_i \neq 0$  do
5:    $q_i \leftarrow r_{i-1} / r_i$ 
6:    $r_{i+1} \leftarrow r_{i-1} - q_i r_i$ 
7:    $s_{i+1} \leftarrow s_{i-1} - q_i s_i$ 
8:    $t_{i+1} \leftarrow t_{i-1} - q_i t_i$ 
9:    $i \leftarrow i + 1$ 
10: end while
11:  $l \leftarrow i - 1$ 
12: return  $r_l, s_l, t_l \in K[x]$ 
```

$$\begin{array}{r} \frac{f}{g} = \frac{g^{-1}f}{g) \quad f} \\ \quad \quad \quad \frac{-f}{0} \end{array}$$

Theorem 2.1.1 (Bezout's identity).

Let $f, g \in K[x]$ such that $f, g \neq 0$

Let $d = \gcd(f, g)$ then $\exists s, t \in K[x]$ such that

$$s_i f + t_i g = d$$

when $\gcd(f, g) = 1 \implies s_l f + t_l g = 1$

$$\implies t_l g \equiv 1 \pmod{f} \implies t_l = \frac{1}{g} \pmod{f}$$

2.1.3 Chinese Remainder Theorem

Theorem 2.1.2. [1] *Let $I_1, I_2, \dots, I_k$ be ideals in a ring R . The map*

$$\begin{aligned} \phi: R &\longrightarrow R/I_1 \times R/I_2 \times \cdots \times R/I_k \\ r &\longmapsto (r + I_1, r + I_2, \dots, r + I_k) \end{aligned}$$

is a ring homomorphism with kernel $I_1 \cap I_2 \cap \cdots \cap I_k$. If for each $i, j \in \{1, 2, \dots, k\}$ with $i \neq j$, the ideals I_i, I_j are called comaximal and the map ϕ is surjective. Moreover,

$$I_1 \cap I_2 \cap \cdots \cap I_k = I_1 I_2 \dots I_k$$

$$\implies R/(I_1 I_2 \dots I_k) = R/(I_1 \cap I_2 \cap \cdots \cap I_k) \cong R/I_1 \times R/I_2 \times \cdots \times R/I_k.$$

2.1.4 Chinese remainder theorem and Interpolation

Theorem 2.1.3. *Let $K[x]$ be a polynomial ring over a field K and let $I \subset K[x]$ be an ideal of the ring then $K[x]/I$ is a field if and only if I is a maximal ideal.*

Given a black box B for a polynomial $f(x) \in K[x]$ and $x = \alpha \in K$ we can evaluate $f(\alpha)$ by querying the black box B at α .

$$\begin{aligned} \phi: K[x] &\longrightarrow K[x]/(x - \alpha) \\ f(x) &\longmapsto f(\alpha) \end{aligned}$$

Thus, evaluating the polynomial at n points is querying the black box at n points $\alpha_1, \dots, \alpha_n$ is the following homomorphism.

$$\begin{aligned}\phi: K[x] &\longrightarrow K[x]/(x - \alpha_1) \times K[x]/(x - \alpha_2) \times \dots \times K[x]/(x - \alpha_n) \\ f(x) &\longmapsto (f(\alpha_1), f(\alpha_2), \dots, f(\alpha_n))\end{aligned}$$

From the Chinese remainder theorem, we know that

$$K[x]/(x - \alpha_1) \times K[x]/(x - \alpha_2) \times \dots \times K[x]/(x - \alpha_n) \cong K[x]/((x - \alpha_1)(x - \alpha_2) \dots (x - \alpha_n))$$

$$\text{Let } [\alpha_1, \dots, \alpha_n] \in K^n$$

such that

$$\alpha_i \neq \alpha_j \forall i \neq j \text{ and } n > 0.$$

$$\text{Let } F(x) = \frac{f(x)}{g(x)} \in K(x), \text{ where } f(x), g(x) \in K[x],$$

$$\text{Let } y_i = F(\alpha_i) = \frac{f(\alpha_i)}{g(\alpha_i)}, \forall 1 \leq i \leq n, \text{ such that } g(\alpha_i) \neq 0.$$

$$\exists! u(x) \in K[x] \text{ of degree } < n \text{ such that } u(\alpha_i) = y_i$$

$$\implies u(x) \equiv y_i \pmod{(x - \alpha_i)}$$

$$\implies F(x) = \frac{f(x)}{g(x)} \equiv u(x) \pmod{(x - \alpha_i)}$$

$$\text{Let } \overline{m}(x) = \prod_{i=1}^n (x - \alpha_i)$$

2.1.5 Main idea

1. We are given the black box of rational polynomial $F(x) \in K(x)$. We have the option to choose n distinct points $\alpha_1, \dots, \alpha_n$, such that $n > \text{degree}_{\text{numerator}} + \text{degree}_{\text{denominator}}$ and evaluate the black box at these points to get $y_1, \dots, y_n$.
2. Construct two polynomials m, u out of these two sets of points.
3. $m(x) = \prod_{i=1}^n (x - \alpha_i)$ is the Chinese remainder theorem polynomial.
4. $u(x) = \text{Interpolation}(\alpha_i, y_i)$. Note that the degree of $u(x) = n - 1$ and the degree of $m(x) = n$.
5. We take these two polynomials m, u and apply the extended euclidean algorithm to get $f(x), g(x) \implies f, g$ appear as remainder and coefficient in the division algorithm.

6. We can think of any rational function $\frac{f}{g}$ as members of an equivalence class. (localization?) with other elements.
7. What MQRFR says is that the $f = r_i$ and $g = t_i$ for the i^{th} iteration of the extended euclidean algorithm such that $\deg(q_i)$ is max.
8. There does seem to be some relationship between Univariate rational functions, the interpolated polynomial u the product polynomial m and the extended euclidean algorithm(m, u).
9. CRT from 265 Dummit and Foote.
- 10.

Chapter 3

Ben-Or Tiwari Interpolation

3.1 Introduction

Ben-Or Tiwari is a two-step multivariate interpolation algorithm that interpolates all the variables of the polynomial simultaneously as opposed to Zippel's multivariate interpolation algorithm, which interpolates the variables one at a time. In the first step, the Ben-Or Tiwari algorithm finds the monomials of the multivariate polynomial and in the second step, it finds the coefficients of the polynomial. Each step of the Ben-Or Tiwari algorithm is based on considering polynomials in different contexts.

First step, uses insights from linear recursive sequences and uses Berlekamp Massey algorithm to recover the monomials, while the second step uses Vandermonde Solver to recover the coefficients.

3.2 Linear Recursive Sequences

Definition 3.2.1 (Linear recurrence sequence). *Any sequence $\{a_n\}$, satisfying a linear homogeneous recurrence with constant coefficients of the form*

$$a_n = c_0 a_{n-k} + c_1 a_{n-k+1} + \cdots + c_{k-1} a_{n-1} \quad \forall n \geq k$$

is called a linear recursive sequence. Where $c_0, c_1, \dots, c_{k-1}$ are constants, and k is the order of the recurrence.

3.2.1 Linear recurrence as Linear System

Linear recurrences have a rich structure that can be explored using concepts from linear algebra. We will see two different way to express linear recurrence as linear systems,

1. Using Companion matrix
2. Using Hankel matrix

Theorem 3.2.1. [3] Given $c_0, c_1, \dots, c_{k-1} \in \mathbb{F}$, let

$$V = \{(a_n) \mid a_n = c_0 a_{n-k} + c_1 a_{n-k+1} + \dots + c_{k-1} a_{n-1}, \forall n \geq k\}$$

denote the vector space of all sequences satisfying the linear recurrence relation determined by $c_0, c_1, \dots, c_{k-1}$. Then the function

$$T : \mathbb{F}^k \rightarrow V$$

defined above is an isomorphism. In particular:

1. $\dim V = k$.
2. If $\{\mathbf{v}_1, \dots, \mathbf{v}_k\}$ is any basis of $\mathbb{F}^k$, then $\{T(\mathbf{v}_1), \dots, T(\mathbf{v}_k)\}$ is a basis of V .

Companion matrix

Consider a linear homogeneous recurrence relation with constant coefficients of order k :

$$a_n = c_0 a_{n-k} + c_1 a_{n-k+1} + \dots + c_{k-1} a_{n-1}$$

We can rewrite this as the following linear transform.

$$C = \begin{bmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \\ c_0 & c_1 & c_2 & \dots & c_{k-1} \end{bmatrix} \begin{bmatrix} a_{n-k} \\ a_{n-k+1} \\ \vdots \\ a_{n-2} \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} a_{n-k+1} \\ a_{n-k+2} \\ \vdots \\ a_{n-1} \\ c_0 a_{n-k} + c_1 a_{n-k+1} + \dots + c_{k-1} a_{n-1} \end{bmatrix}$$

where C is called the **companion matrix**.

This setup looks similar to a dynamic system, if we define the initial state of the system as $\mathbf{v}_0$ with the initial values of the sequence, then $\mathbf{v}_0 = [a_0, a_1, \dots, a_{k-1}]^T$. Then we can define the state of the system at step i as

$$\mathbf{v}_i = C^i \mathbf{v}_0$$

Where $\mathbf{v}_i = [a_i, a_{i+1}, \dots, a_{i+k-1}]^T$. The recurrence can be written in matrix form:

$$\mathbf{v}_{i+1} = C \mathbf{v}_i,$$

3.2.2 Characteristic Polynomial of the Companion Matrix

Claim 1. *The characteristic polynomial of the companion matrix*

$$C = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ c_0 & c_1 & c_2 & \cdots & c_{k-1} \end{bmatrix}_{k \times k}$$

is given by

$$\lambda^k - c_{k-1}\lambda^{k-1} - c_{k-2}\lambda^{k-2} - \cdots - c_1\lambda - c_0$$

Proof.

$$M_k(\lambda) := \lambda I - C = \begin{bmatrix} \lambda & -1 & 0 & \cdots & 0 \\ 0 & \lambda & -1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -1 \\ -c_0 & -c_1 & -c_2 & \cdots & \lambda - c_{k-1} \end{bmatrix}$$

Expanding along the first column Column 1 has only two nonzero entries, so

$$\Delta_k(\lambda) = \lambda \det M_k^{(1,1)} + (-c_0)(-1)^{k+1} \det M_k^{(k,1)},$$

where $M_k^{(i,j)}$ denotes the minor formed by deleting row i and column j .

The $(k, 1)$ -minor. Deleting row k and column 1 leaves a lower-bidiagonal matrix with diagonal entries -1 :

$$M_k^{(k,1)} = \begin{bmatrix} -1 & 0 & \cdots & 0 \\ \lambda & -1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & -1 \end{bmatrix}, \quad \Rightarrow \quad \det M_k^{(k,1)} = (-1)^{k-1}.$$

Hence the second term equals

$$(-c_0)(-1)^{k+1}(-1)^{k-1} = -c_0.$$

The (1, 1)-minor. Deleting row 1, column 1 yields

$$M_k^{(1,1)} = \begin{bmatrix} \lambda & -1 & 0 & \cdots & 0 \\ 0 & \lambda & -1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -1 \\ -c_1 & -c_2 & -c_3 & \cdots & \lambda - c_{k-1} \end{bmatrix}.$$

We now evaluate $\det M_k^{(1,1)}$ inductively.

Using Induction to evaluate $\det M_k^{(1,1)}$

For $m \geq 1$, define

$$B_m(\lambda) = \begin{bmatrix} \lambda & -1 & 0 & \cdots & 0 \\ 0 & \lambda & -1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & -1 \\ -d_1 & -d_2 & -d_3 & \cdots & \lambda - d_m \end{bmatrix}.$$

Claim. $\det B_m(\lambda) = \lambda^m - d_m \lambda^{m-1} - d_{m-1} \lambda^{m-2} - \cdots - d_2 \lambda - d_1$.

Proof by induction.

Base case: $m = 1$: $\det[\lambda - d_1] = \lambda - d_1$ (true).

Inductive case: Assume the formula holds for $m - 1$. Expanding $\det B_m$ along the first column:

$$\det B_m(\lambda) = \lambda \det B_{m-1}(\lambda) + (-d_1)(-1)^{m+1} \det T_{m-1},$$

where

$$B_{m-1} = \begin{bmatrix} \lambda & -1 & 0 & \cdots & 0 & 0 \\ 0 & \lambda & -1 & \ddots & 0 & 0 \\ \vdots & \vdots & \vdots & \cdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & \lambda & -1 \\ -d_2 & -d_3 & -d_4 & \cdots & -d_{m-1} & \lambda - d_m \end{bmatrix} \quad \text{and} \quad T_{m-1} = \begin{bmatrix} -1 & 0 & 0 & \cdots & 0 & 0 \\ \lambda & -1 & 0 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \cdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & -1 & 0 \\ 0 & 0 & 0 & \cdots & \lambda & -1 \end{bmatrix}$$

T_{m-1} is lower-bidiagonal with diagonal entries -1 , hence $\det T_{m-1} = (-1)^{m-1}$. Thus the second term is $(-d_1)(-1)^{m+1}(-1)^{m-1} = -d_1$.

By the induction hypothesis,

$$\det B_{m-1}(\lambda) = \lambda^{m-1} - d_m \lambda^{m-2} - \cdots - d_3 \lambda - d_2$$

$$\det B_m(\lambda) = \lambda(\lambda^{m-1} - d_m \lambda^{m-2} - \cdots - d_3 \lambda - d_2) - d_1$$

$$\det B_m(\lambda) = \lambda^m - d_m \lambda^{m-1} - \dots - d_3 \lambda^2 - d_2 \lambda - d_1$$

Applying the claim with $m = k - 1$ for $(d_1, \dots, d_{k-1}) = (c_1, \dots, c_{k-1})$ gives us

$$\det M_k^{(1,1)} = \lambda^{k-1} - c_{k-1} \lambda^{k-2} - \dots - c_2 \lambda - c_1.$$

Substituting $M_k^{(1,1)}$ in $\Delta_k(\lambda) = \lambda \det M_k^{(1,1)} + (-c_0)(-1)^{k+1} \det M_k^{(k,1)}$

$$\begin{aligned} \Delta_k(\lambda) &= \lambda(\lambda^{k-1} - c_{k-1} \lambda^{k-2} - \dots - c_2 \lambda - c_1) - c_0 \\ &= \lambda^k - c_{k-1} \lambda^{k-1} - c_{k-2} \lambda^{k-2} - \dots - c_1 \lambda - c_0 \end{aligned}$$

□

A general solution is a basis that generates each element of the recurrence relation as a linear combination.

3.2.3 Eigen values of the Companion matrix are the general solution of the linear recurrence

Given a linear homogeneous linear recurrence relation with constant coefficients of order k :

$$a_n = c_0 a_{n-k} + c_1 a_{n-k+1} + \dots + c_{k-1} a_{n-1}$$

The characteristic polynomial $\Lambda(\lambda)$ of the companion matrix C is given by

$$\lambda^k - c_{k-1} \lambda^{k-1} - c_{k-2} \lambda^{k-2} - \dots - c_1 \lambda - c_0 = 0$$

Let the characteristic polynomial have k distinct roots $\lambda_1, \dots, \lambda_k$, then since C is a square matrix with distinct eigen values, C is diagonalizable,

$$C = PDP^{-1} \text{ Where } D = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_k)$$

We saw earlier that we can formulate linear recurrence as the following dynamic system.

$$v_i = C^i v_0 \implies v_i = (PD^i P^{-1}) v_0$$

Where $D^i = \text{diag}(\lambda_1^i, \lambda_2^i, \dots, \lambda_k^i)$, and $\mathbf{v}_i = [a_i, a_{i+1}, \dots, a_{i+k-1}]^T$. Thus, we can see that the companion matrix acts as a shift operator for the input vector v_0 .

Constructing Matrix P

Let the characteristic polynomial of the companion matrix C of rank k have k distinct roots $\lambda_1, \dots, \lambda_k$.

$$w = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{k-1} \end{bmatrix} \text{ be an eigen vector of } C \implies Cw = \lambda w$$

Since the companion matrix adds as a shift operator, we have

$$Cw = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ c_0 & c_1 & c_2 & \cdots & c_{k-1} \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_{k-2} \\ w_{k-1} \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_{k-1} \\ c_0 w_0 + c_1 w_1 + \cdots + c_{k-1} w_{k-1} \end{bmatrix} = \begin{bmatrix} \lambda w_0 \\ \lambda w_1 \\ \vdots \\ \lambda w_{k-2} \\ \lambda w_{k-1} \end{bmatrix}$$

For the first $k-1$ entries of Cw we can see that

$$w_{i+1} = \lambda w_i$$

$$\implies w_1 = \lambda w_0, w_2 = \lambda w_1 = \lambda^2 w_0, w_3 = \lambda^3 w_0, \dots, w_{k-1} = \lambda^{k-1} w_0$$

For the last entry of Cw , we have

$$c_0 w_0 + c_1 w_1 + \cdots + c_{k-1} w_{k-1} = \lambda w_{k-1}$$

Substituting $w_i = \lambda^i w_0$ in above equation,

$$c_0 w_0 + c_1 \lambda w_0 + \cdots + c_{k-1} \lambda^{k-1} w_0 = \lambda (\lambda^{k-1} w_0) = \lambda^k w_0$$

Dividing both sides by w_0 , gives us the characteristic polynomial

$$\lambda^k - c_{k-1} \lambda^{k-1} - c_{k-2} \lambda^{k-2} - \cdots - c_1 \lambda - c_0 = 0$$

$$\implies w_0 = 1, w_1 = \lambda, w_2 = \lambda^2, \dots, w_{k-1} = \lambda^{k-1},$$

Thus, the eigenvector

$$w(\lambda) = \begin{bmatrix} 1 \\ \lambda \\ \lambda^2 \\ \vdots \\ \lambda^{k-1} \end{bmatrix}$$

Diagonalization matrix P

$$P = [w(\lambda_1) \ \cdots \ w(\lambda_k)] = \begin{bmatrix} 1 & \cdots & 1 \\ \lambda_1 & \cdots & \lambda_k \\ \vdots & \ddots & \vdots \\ \lambda_1^{k-1} & \cdots & \lambda_k^{k-1} \end{bmatrix}, \quad C = P \operatorname{diag}(\lambda_1, \dots, \lambda_k) P^{-1}.$$

3.2.4 Hankel Matrix

What if we don't know the linear recurrence, but we can generate infinitely many terms of the sequence. How to find the characteristic polynomial in this case? For this we need Hankel Matrix.

Definition 3.2.2. *Given a sequence $\{a_n\}$ the corresponding Hankel matrix is given by*

$$H_m = \begin{bmatrix} a_0 & a_1 & a_2 & \cdots & a_{m-1} \\ a_1 & a_2 & a_3 & \cdots & a_m \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m-1} & a_m & a_{m+1} & \cdots & a_{2m-2} \end{bmatrix}_{m \times m}$$

The Hankel matrix H_m can be an infinite matrix if use infinitely many terms of the sequence to construct it.

Theorem 3.2.2 (Kronecker-Hankel Theorem [2]). *A sequence of numbers satisfies a linear recurrence relation if and only if the associated infinite Hankel matrix has finite rank. The order of the linear recurrence is equal to the rank of the Hankel matrix.*

Consequence of Kronecker-Hankel theorem

If H_m is an infinite Hankel matrix of rank $k \implies \exists$ a submatrix H_k such that

$$H_k = \begin{bmatrix} v_0^T \\ [Cv_0]^T \\ \vdots \\ [C^{k-1}v_0]^T \end{bmatrix} \quad \text{where } v_0 = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{k-1} \end{bmatrix} \quad \text{and } C = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ c_0 & c_1 & c_2 & \cdots & c_{k-1} \end{bmatrix}$$

3.2.5 Polynomials as linear recurrences.

Lemma 3.2.3. *Let $\mathbb{F}[x]$ be a ring of polynomials over field $\mathbb{F}$, let $f(x) \in \mathbb{F}[x]$, and $\alpha \in \mathbb{F}$ be some arbitrary element of $\mathbb{F}$, then the sequence*

$$\{f(1), f(\alpha), f(\alpha^2), \dots, f(\alpha^n)\}$$

is a linear recursive sequence.

Proof. Let

$$f(x) = b_{k-1}x^{k-1} + b_{k-2}x^{k-2} + \dots + b_1x + b_0 \in \mathbb{F}[x],$$

where $\mathbb{F}$ is a field. For some arbitrary $\alpha \in \mathbb{F}$, we define the sequence $\{a_n\}$ such that

$$a_i = f(\alpha^i) = \sum_{j=0}^{k-1} b_j(\alpha^i)^j = \sum_{j=0}^{k-1} b_j(\alpha^j)^i.$$

Hence, each a_i is a linear combination of the geometric sequences $(\alpha^j)^i$ for $j = 0, 1, \dots, k-1$.

The corresponding $m \times m$ Hankel matrix associated with the sequence $\{a_i\}$ is

$$H_m = \begin{bmatrix} a_0 & a_1 & a_2 & \cdots & a_{m-1} \\ a_1 & a_2 & a_3 & \cdots & a_m \\ a_2 & a_3 & a_4 & \cdots & a_{m+1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m-1} & a_m & a_{m+1} & \cdots & a_{2m-2} \end{bmatrix}.$$

Using Rank decomposition, we can factorize the Hankel matrix as follows,

$$H_m = V_m D V_m^\top.$$

Where,

1. Vandermonde-type matrix V_m :

$$V_m = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ \alpha^0 & \alpha^1 & \alpha^2 & \cdots & \alpha^{k-1} \\ (\alpha^0)^2 & (\alpha^1)^2 & (\alpha^2)^2 & \cdots & (\alpha^{k-1})^2 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ (\alpha^0)^{m-1} & (\alpha^1)^{m-1} & (\alpha^2)^{m-1} & \cdots & (\alpha^{k-1})^{m-1} \end{bmatrix}_{m \times k}.$$

2. Diagonal matrix D :

$$D = \text{diag}(b_0, b_1, \dots, b_{k-1}) = \begin{bmatrix} b_0 & 0 & 0 & \cdots & 0 \\ 0 & b_1 & 0 & \cdots & 0 \\ 0 & 0 & b_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & b_{k-1} \end{bmatrix}_{k \times k}.$$

3. Transpose $V_m^\top$:

$$V_m^\top = \begin{bmatrix} 1 & \alpha^0 & (\alpha^0)^2 & \cdots & (\alpha^0)^{m-1} \\ 1 & \alpha^1 & (\alpha^1)^2 & \cdots & (\alpha^1)^{m-1} \\ 1 & \alpha^2 & (\alpha^2)^2 & \cdots & (\alpha^2)^{m-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{k-1} & (\alpha^{k-1})^2 & \cdots & (\alpha^{k-1})^{m-1} \end{bmatrix}_{k \times m}.$$

Each entry of H_m satisfies

$$a_{p+q} = \sum_{j=0}^{k-1} b_j (\alpha^j)^{p+q} = \sum_{j=0}^{k-1} b_j (\alpha^j)^p (\alpha^j)^q.$$

The (p, q) -entry of $V_m D V_m^\top$ is

$$(V_m D V_m^\top)_{p,q} = \sum_{j=0}^{k-1} (V_m)_{p,j} D_{j,j} (V_m^\top)_{j,q} = \sum_{j=0}^{k-1} b_j (\alpha^j)^p (\alpha^j)^q = a_{p+q}.$$

Thus, $H_m = V_m D V_m^\top$.

Since V_m is an $m \times k$ matrix, we have

$$\text{rank}(V_m) \leq k,$$

$$\implies \text{rank}(H_m) = \text{rank}(V_m D V_m^\top) \leq \text{rank}(V_m) \leq k.$$

Now for $m = k + 1$,

$$\text{rank}(H_{k+1}) \leq k < k + 1,$$

so

$$\det(H_{k+1}) = 0$$

By theorem 3.2.2 we can say that the entries of the matrix H_k , the terms of the sequence $\{f(\alpha^i)\}$ form a linear recurrence.

□

The above argument can be extended to multivariate polynomials as well

3.2.6 Hankel matrix with Gaussian elimination

3.2.7 Berlekamp Massey Algorithm

The Berlekamp Massey Algorithm is an iterative algorithm that finds the shortest Linear Feedback Shift Register. It is used in decoding algebraic error-correcting codes, such as BCH codes and Reed–Solomon code.

3.2.8 Equivalence of Berlekamp Massey Algorithm and Extended Euclidean Algorithm

3.3 Polynomials as vectors

Let $\mathbb{F}$ be a field, we know that the ring of polynomials $\mathbb{F}[x]$ is an also infinite dimensional vector space, with the set of monomial basis $\{1, x, x^2, \dots\}$, as one of the basis. Let $\mathbb{F}^{n+1}$ be an $n + 1$ dimensional subspace of $\mathbb{F}[x]$, then we have the following result-

$$\mathbb{F}^{n+1} \sim \mathbb{F}[x]/(x^{n+1})$$

We can define a vector $\vec{x} \in \mathbb{F}^{n+1}$ such that

$$\vec{x} = \begin{bmatrix} x^n \\ x^{n-1} \\ \vdots \\ x \\ 1 \end{bmatrix}$$

for some $x \in \mathbb{F}$. Let $\vec{a} \in \mathbb{F}^{n+1}$ be some arbitrary such that

$$\vec{a} = \begin{bmatrix} a_n \\ a_{n-1} \\ \vdots \\ a_1 \\ a_0 \end{bmatrix}$$

such that

$$a(x) = a_n x^n + a_{n-1} x^{n-1} + a_{n-2} x^{n-2} + \dots + a_1 x + a_0$$

We can rewrite the polynomial a as an inner product of a coefficient vector and the monomial vector.

$$\begin{bmatrix} a_n \\ a_{n-1} \\ \vdots \\ a_1 \\ a_0 \end{bmatrix} \begin{bmatrix} x^n & x^{n-1} & \dots & x & 1 \end{bmatrix} = a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0$$

Lemma 3.3.1. *Let $\mathbb{F}[x_1, \dots, x_m]$ be a ring of polynomials over field $\mathbb{F}$, let $f(x_1, \dots, x_m) \in \mathbb{F}[x_1, \dots, x_m]$, and $\alpha_1, \dots, \alpha_m \in \mathbb{F}^m$ be some arbitrary element, then the sequence*

$$\{f(1, \dots, 1), f(\alpha_1, \dots, \alpha_m), f(\alpha_1^2, \dots, \alpha_m^2), \dots, f(\alpha_1^n, \dots, \alpha_m^n)\}$$

is a linear recursive sequence.

Proof.

□

Lemma 3.3.2. *The roots of the characteristic polynomial encapsulate the structure of the monomials of the polynomial.*

3.4 Interpolation as a linear map

$$a_n x^n + a_{n-1} x^{n-1} + a_{n-2} x^{n-2} + \dots + a_1 x + a_0 \in \mathbb{F}[x].$$

Let $\vec{\alpha}, \vec{b} \in \mathbb{F}^d$.

$$\begin{bmatrix} 1 & \alpha_1 & \dots & \alpha_1^d \\ 1 & \alpha_2 & \dots & \alpha_2^d \\ \vdots & \vdots & \dots & \vdots \\ 1 & \alpha_d & \dots & \alpha_d^d \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_d \end{bmatrix} = \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_d \end{bmatrix}$$

$$a_0 + a_1 \alpha_1 + a_2 \alpha_1^2 + \dots + a_d \alpha_d^d = b_0$$

3.4.1 Zippel Transposed Vandermonde Solver

3.4.2 Fast Transposed Vandermonde Solver

3.5 Ben-Or and Tiwari's Multivariate Polynomial Interpolation

Algorithm 2 Ben-Or & Tiwari Multivariate Interpolation

Require: Black-box function f (evaluates the target polynomial), number of variables n , degree bound D , finite field $\mathbb{Z}(p)$

Ensure: Sparse polynomial $P(x_1, x_2, \dots, x_n)$ as a list of [coefficient, exponents] pairs.

```
1:  $T_{\max} \leftarrow (D + 1)^n$  ▷ Upper bound on number of terms
2:  $m \leftarrow 2 \cdot T_{\max}$  ▷ Number of evaluation points
3: Step 2: Generate evaluation points
4:  $evaluations \leftarrow$  empty list
5: for  $k \leftarrow 1$  to  $m$  do
6:    $x_k \leftarrow [\alpha^{k^i} \text{ for } i = 1 \dots n]$ 
7:    $v_k \leftarrow f(x_k)$ 
8:   Append  $v_k$  to  $evaluations$ 
9: end for
10: Step 3: Construct minimum characteristic polynomial using Berlekamp-Massey algorithm
11:  $\Lambda \leftarrow Berlekamp\_Massey(evaluations, p)$ 
12: Step 5: Factor  $\Lambda(z)$  to find roots
13:  $roots \leftarrow \text{FIND\_ROOTS}(\Lambda)$ 
14: Step 6: Recover exponents from roots
15:  $terms \leftarrow$  empty list
16: for each root in  $roots$  do
17:    $exponents \leftarrow \text{TRIAL\_DIVISION}(root, n)$ 
18:   Append  $exponents$  to  $terms$ 
19: end for
20: Step 7: Recover coefficients via linear system
21:  $A \leftarrow \text{CONSTRUCT\_VANDERMONDE\_MATRIX}(terms, evaluations)$ 
22:  $b \leftarrow evaluations[1 \dots T_{\max}]$ 
23:  $coefficients \leftarrow \text{SOLVE\_LINEAR\_SYSTEM}(A, b)$ 
24: Step 8: Validate with additional evaluations
25:  $validated \leftarrow \text{TRUE}$ 
26: for  $k \leftarrow m + 1$  to  $m + 10$  do
27:    $x_k \leftarrow [f(\alpha)^{k^i} \text{ for } i = 1 \dots n]$ 
28:   if  $f(x_k) \neq \text{EVALUATE\_POLYNOMIAL}(terms, coefficients, x_k)$  then
29:      $validated \leftarrow \text{FALSE}$ 
30:     break
31:   end if
32: end for
33: if not  $validated$  then
34:   Increase  $T_{\max}$  and restart
35: end if
36: return  $\text{SPARSE\_POLYNOMIAL}(terms, coefficients)$ 
```

Chapter 4

Multivariate Rational function interpolation

4.1 Kaltofen Yang algorithm

We saw in chapter 2 that extended euclidean algorithm is crucial in order to recover the numerator and the denominator, this works well for ring of univariate polynomials that is an Euclidean domain, however the ring of multivariate polynomials is not an Euclidean domain

$$\begin{array}{ccc}
 \mathbb{Z}_p(x_1, \dots, x_n) & \xrightarrow{\phi} & \mathbb{Z}_p(x) \\
 \downarrow & & \downarrow \\
 \mathbb{Z}_p(x_1, \dots, x_n)/(x_1 - \alpha_j, \dots, x_i - \beta_i(\alpha_j - \sigma_1) + \sigma_i) \cong \mathbb{Z}_p & & \mathbb{Z}_p(x)/(x - \alpha_j) \cong \mathbb{Z}_p
 \end{array}$$

Where,

$$\begin{array}{ccc}
 \phi: \mathbb{Z}_p(x_1, \dots, x_2) & \longrightarrow & \mathbb{Z}_p[x] \\
 x_1 & \longmapsto & x \\
 x_2 & \longmapsto & \beta_2(x - \sigma_1) + \sigma_2 \\
 \vdots & & \vdots \\
 x_n & \longmapsto & \beta_n(x - \sigma_1) + \sigma_n
 \end{array}$$

$$T(x) = \frac{f(x)}{g(x)} = \frac{ff(x, \beta_2(x - \sigma_1) + \sigma_2, \dots, \beta_n(x - \sigma_1) + \sigma_n)}{gg(x, \beta_2(x - \sigma_1) + \sigma_2, \dots, \beta_n(x - \sigma_1) + \sigma_n)}$$

Multivariate Rational Function Interpolation

```

1: Input: Modular black box  $B : (\mathbb{Z}^n, p) \rightarrow \mathbb{Z}_p$  for a rational function  $\frac{ff(x_1, \dots, x_n)}{gg(x_1, \dots, x_n)}$ 
   over  $\mathbb{Z}_p$ , where  $ff, gg \in \mathbb{Z}[x_1, \dots, x_n]$ ,  $\gcd(ff, gg) = 1$ ,  $gg$  is monic, prime  $p$ , number
   of variables  $n$ , and list of variables  $vars$ .
2: Output:  $ff(x_1, \dots, x_n), gg(x_1, \dots, x_n)$  or FAIL.
3:  $Primes \leftarrow [2, 3, \dots, p_n] \in \mathbb{Z}^n$ 
4: Pick random vector  $\beta = [\beta_2, \dots, \beta_n] \in \mathbb{Z}_p^{n-1}$ 
5:  $num \leftarrow [], den \leftarrow []$ 
6:  $num\_points\_mqrfr \leftarrow 0$ 
7:  $numerator\_done \leftarrow \text{false}$ 
8:  $denominator\_done \leftarrow \text{false}$ 
9:  $T\_old \leftarrow 1$ 
10:  $T \leftarrow 4$ 
11:  $\sigma^0 \leftarrow [1, \dots, 1] \in \mathbb{Z}_p^n$ 
12:  $(f_0, g_0) \leftarrow \text{NDSA}(B, \sigma^0, \beta, p, T)$  where,  $f_0, g_0 \in \mathbb{Z}_p[x]$ 
13:  $num.append(f_0(\sigma_1^0) \bmod p)$ 
14:  $den.append(g_0(\sigma_1^0) \bmod p)$ 
15:  $num\_points\_mqrfr \leftarrow \deg(f_0) + \deg(g_0) + 2$ 
16: while true do
17:   for  $j \leftarrow T\_old$  to  $2T - 1$  do
18:      $\sigma^j \leftarrow [2^j, 3^j, \dots, p_n^j] \bmod p$ 
19:      $(f_j, g_j) \leftarrow \text{NDSA}(B, \sigma^j, \beta, p, num\_points\_mqrfr)$ 
20:     if not  $numerator\_done$  then
21:        $num.append(f_j(\sigma_1^j) \bmod p)$ 
22:     end if
23:     if not  $denominator\_done$  then
24:        $den.append(g_j(\sigma_1^j) \bmod p)$ 
25:     end if
26:   end for
   Construct minimum characteristic polynomials using Berlekamp-Massey algo-
   rithm
27:   if not  $numerator\_done$  then
28:      $\Lambda_{num}(z) \leftarrow \text{Berlekamp\_Massey}(num, p, z) \in \mathbb{Z}_p[z]$ 
29:      $s \leftarrow \deg(\Lambda_{num}(z))$ 
30:     Let  $roots_{num}$  be the distinct roots of  $\Lambda_{num}(z) \in \mathbb{Z}_p[z]$ .
31:   end if
32:   if not  $denominator\_done$  then
33:      $\Lambda_{den}(z) \leftarrow \text{Berlekamp\_Massey}(den, p, z) \in \mathbb{Z}_p[z]$ 

```

```

34:      $d \leftarrow \deg(\Lambda_{den}(z))$ 
35:     Let  $roots_{den}$  be the distinct roots of  $\Lambda_{den}(z) \in \mathbb{Z}_p[z]$ .
36: end if
37: if  $s = |roots_{num}|$  and  $s < T$  then
38:      $numerator\_done \leftarrow \text{true}$ 
39: end if
40: if  $d = |roots_{den}|$  and  $d < T$  then
41:      $denominator\_done \leftarrow \text{true}$ 
42: end if
43: if  $numerator\_done \wedge denominator\_done$  then
44:     break
45: end if
46:  $T\_old \leftarrow 2T$ 
47:  $T \leftarrow 2T$ 
48: end while
    Recover monomials from roots using trial division
49:  $N \leftarrow \text{get\_monomial}(roots_{num}, Primes, n, vars)$ 
50:  $D \leftarrow \text{get\_monomial}(roots_{den}, Primes, n, vars)$ 
51: if  $N = \text{FAIL}$  or  $D = \text{FAIL}$  then
52:     return FAIL
53: end if
54: Recover coefficients via Zippel Vandermonde solver
55:  $A \leftarrow \text{Zippel\_Vandermonde\_solver}(num, s, roots_{num}, \Lambda_{num}(z), p)$ 
56:  $B \leftarrow \text{Zippel\_Vandermonde\_solver}(den, d, roots_{den}, \Lambda_{den}(z), p)$ 
57:  $ff \leftarrow \sum_{m=1}^s A_m N_m, \quad gg \leftarrow \sum_{m=1}^d B_m D_m$ 
58: Let  $\mu$  be the leading coefficient of  $gg$  in grlex order where  $x_1 > \dots > x_n$ .
59:  $ff \leftarrow \mu^{-1} ff \bmod p, \quad gg \leftarrow \mu^{-1} gg \bmod p$ .
60: return  $ff, gg$ .

```

Numerator Denominator Separation Algorithm (NDSA)

- 1: **Input:** Modular black box B for rational function $\frac{ff(x_1, \dots, x_n)}{gg(x_1, \dots, x_n)}$ over $\mathbb{Z}_p$, prime p , where $ff, gg \in K[x_1, \dots, x_n]$, $\gcd(ff, gg) = 1$, $\sigma \in \mathbb{Z}_p^n$, $\beta \in \mathbb{Z}_p^{n-1}$, $num_points \in \mathbb{N}$.
- 2: **Output:** $f(x) = \frac{ff(x, \beta_2(x-\sigma_1)+\sigma_2, \dots, \beta_n(x-\sigma_1)+\sigma_n)}{c}$, $g(x) = \frac{gg(x, \beta_2(x-\sigma_1)+\sigma_2, \dots, \beta_n(x-\sigma_1)+\sigma_n)}{c}$, for some scalar $c \in \mathbb{Z}_p$.
- 3: $t \leftarrow num_points$
- 4: **while** true **do**
- 5: Pick random vector $\alpha = [\alpha_1, \dots, \alpha_t] \in \mathbb{Z}_p^t$
- 6: $m(x) \leftarrow \prod_{k=1}^t (x - \alpha_k) \in \mathbb{Z}_p[x]$
- 7: $\Phi \leftarrow [\phi(\alpha_1), \dots, \phi(\alpha_t)] \in \mathbb{Z}_p^{t \times n}$ such that: $\phi(\alpha_k) \leftarrow [\alpha_k, \beta_2(\alpha_k - \sigma_1) + \sigma_2, \dots, \beta_n(\alpha_k - \sigma_1) + \sigma_n] \bmod p \ \forall 1 \leq k \leq t$
- 8: $Y \leftarrow [B(\Phi(\alpha_1), p), \dots, B(\Phi(\alpha_t), p)] \in \mathbb{Z}_p^t$
- 9: $u(x) \leftarrow \text{Interpolate}(\alpha, Y, x) \bmod p$
- 10: $(f(x), g(x), deg_q) \leftarrow \text{MQRFR}(m, u) \bmod p$
- 11: **if** $deg_q > 1$ **then**
- 12: **break**
- 13: **else**
- 14: $t \leftarrow 2t$
- 15: **end if**
- 16: **end while**
- 17: **return** f, g

Chapter 5

Applications

Bibliography

- [1] D.S. Dummit and R.M. Foote. *Abstract Algebra*. Wiley, 2003.
- [2] Shuai Geng, Jie Zhang, and Yuanming Zhang. On hankel operators on mixed norm spaces and a related operator equation. *arXiv preprint arXiv:2206.04126*, 2022.
- [3] Nicholson. 7.5: More on linear recurrences. [https://math.libretexts.org/Bookshelves/Linear_Algebra/Linear_Algebra_with_Applications_\(Nicholson\)/07:Linear_Transformations/7.05:More_on_Linear_Recurrences](https://math.libretexts.org/Bookshelves/Linear_Algebra/Linear_Algebra_with_Applications_(Nicholson)/07:Linear_Transformations/7.05:More_on_Linear_Recurrences), 2023. Part of the book "Linear Algebra with Applications (Nicholson)", Mathematics LibreTexts, Page ID: 58873.

Appendix A

Code

Appendices should be used for supplemental information that does not form part of the main research. Remember that figures and tables in appendices should not be listed in the List of Figures or List of Tables.